

*bKash presents*

# SUST CSE Carnival 2026

Shahjalal University of Science & Technology

---

**Codex Community Hackathon**

## **Workshop Task Instructions**

Hackathon Challenge — Ticket Analyzer

*Build it with Puku • Deploy it to the cloud • Document it*

Organized by SUST CSE Society

Hackathon Sponsor: Poridhi.io • Title Sponsor: bKash

[csecarnival.sust.edu](https://csecarnival.sust.edu)

## How to Use This Document

This is your task-instruction sheet for the first workshop of bKash presents SUST CSE Carnival 2026 — Codex Community Hackathon. It tells you what to build, the workflow to follow, and how your submission will be judged.

The product you are building is defined by a minimal Product Requirement Document (PRD). The full PRD is reproduced for your convenience as Appendix A — Reference PRD at the end of this document. Read the three tasks below first, then treat the PRD as your single source of truth for scope, API contracts, data model, and acceptance criteria.

Your entire build-test-deploy workflow runs through Puku Editor and the Puku CLI, with cloud credentials and the deployment VM provided through your Poridhi Lab environment. You may also target AWS directly using credentials issued through the Poridhi Lab.

**Puku Editor / CLI:** <https://puku.sh>

**Poridhi Lab:** <https://poridhi.io>

**Event site:** <https://csecarnival.sust.edu/>

## The Challenge at a Glance

| Field             | Detail                                                                   |
|-------------------|--------------------------------------------------------------------------|
| Event             | bKash presents SUST CSE Carnival 2026 — Codex Community Hackathon        |
| Project           | Ticket Analyzer — a minimal full-stack app (see Appendix A)              |
| Core stack        | React/Vite, FastAPI, tiny Hugging Face model, PostgreSQL, Docker Compose |
| Primary tool      | Puku Editor & Puku CLI for build, run, test, troubleshoot, and deploy    |
| Deployment target | AWS or Poridhi Cloud VM (credentials from the Poridhi Lab)               |
| Deliverables      | Working codebase, live deployment, and project documentation             |

## Your Tasks

Complete the following three tasks in order. Each builds on the previous one.

### Task 1 — Build, Run & Troubleshoot Locally with Puku

Using the attached minimal PRD (Appendix A) as your specification, create the full project codebase through Puku Editor / Puku CLI, then run, test, and debug it locally — all inside the Puku workflow.

#### What to do:

1. Open Puku Editor and start a new project workspace for the Ticket Analyzer.

2. Use Puku to scaffold the codebase from the PRD: the React/Vite frontend, the FastAPI backend, the tiny Hugging Face sentiment model integration, and the PostgreSQL data layer.
3. Build the repository to match the structure, API contract, data model, and environment variables defined in the PRD.
4. Write the Dockerfiles for frontend and backend, plus the docker-compose.yml for local orchestration.
5. Deploy and run the project locally through the Puku CLI (e.g. building images and bringing the stack up with docker compose).
6. Test the application end to end: open the frontend, submit a ticket, confirm sentiment is returned, and verify the ticket persists in PostgreSQL after a page refresh.
7. When something breaks, troubleshoot it using the Puku CLI — inspect logs, debug containers, and iterate until the app runs cleanly.

#### **Definition of done for Task 1:**

- The full stack runs locally via Puku with a single compose command.
- GET /health returns ok; POST /tickets and GET /tickets behave per the PRD.
- A submitted ticket is analyzed, saved, and still visible after refresh.

## **Task 2 — Deploy to AWS or Poridhi Cloud VM via Puku CLI**

Once the app works locally, deploy it to the cloud using the Puku CLI. You may target either AWS or a Poridhi Cloud VM. The cloud credentials you need are issued through your Poridhi Lab environment.

#### **What to do:**

1. Open your provided Poridhi Lab environment and obtain the AWS credentials (access key / secret, region, and any session details) issued there.
2. Configure those credentials for the Puku CLI so it can authenticate to the cloud on your behalf. Never hard-code secrets into the repository — use environment variables or the credential mechanism Puku expects. You can use aws-cli for setting up the credentials first.
3. Push your source code to GitHub and your built frontend and backend images to DockerHub.
4. Use the Puku CLI to provision / connect to the target (an AWS instance or the Poridhi Cloud VM) and deploy the stack there with Docker Compose using your published images.
5. Verify the deployment from the public endpoint: the frontend loads, the health check passes, and tickets submitted on the deployed app are analyzed and persisted.

#### **Definition of done for Task 2:**

- Code is on GitHub; frontend and backend images are on DockerHub.
- AWS credentials were sourced from the Poridhi Lab and used securely (no secrets committed).
- The application is reachable on the AWS or Poridhi Cloud VM endpoint and passes the same end-to-end test as the local run.

## Task 3 — Document the Project

Produce clear project documentation so anyone can understand, run, and deploy your Ticket Analyzer.

**Your documentation must cover:**

- Overview: what the project does and the architecture (frontend → backend → model + database).
- Local setup: prerequisites, environment variables, and the exact Puku / compose commands to run it locally.
- API reference: the GET /health, POST /tickets, and GET /tickets endpoints with example request and response bodies.
- Deployment guide: how the app was deployed to AWS / Poridhi Cloud VM via the Puku CLI, including how credentials are supplied (without exposing real secrets).
- Links: the GitHub repository, the DockerHub images, and the live deployed URL.
- Troubleshooting notes: issues you hit and how you resolved them with Puku.

A README.md in the repository root is the expected home for this documentation, matching the PRD repository structure.

**Definition of done for Task 3:**

- A complete README (and PRD.md) is committed to the repository.
- A new reader can clone, run locally, and redeploy by following your docs alone.

## Overall Definition of Done

Your submission is complete when every item below is true. These map directly to the PRD acceptance criteria.

- Frontend opens successfully (locally and on the deployed endpoint).
- GET /health returns status ok.
- A user can submit a ticket from the frontend.
- The backend analyzes ticket sentiment with the tiny Hugging Face model.
- Each ticket is saved in PostgreSQL and survives a page refresh.
- Source code is on GitHub and images are on DockerHub.
- The app runs on AWS or the Poridhi Cloud VM, deployed via Puku CLI.
- Project documentation is complete and accurate.

## Tips

- Let Puku do the heavy lifting — scaffold, run, and debug through the editor/CLI rather than fighting tooling by hand.
- Pre-build and push the backend image early; the model download can be slow during a live run.
- Keep secrets out of Git. Pull AWS credentials from the Poridhi Lab at deploy time only.
- Test the deployed URL, not just localhost — the demo is judged on the live app.

# Appendix A — Reference PRD (Ticket Analyzer)

The following is the original minimal Product Requirement Document. It is the authoritative specification for what you build in the tasks above.

## Product Requirement Document

### Ticket Analyzer

Minimal full-stack engineering demo

| Field      | Detail                                                                   |
|------------|--------------------------------------------------------------------------|
| Workshop   | FDE Capability Buildup                                                   |
| Demo Goal  | Build locally, containerize, push, and deploy on Poridhi Lab VM          |
| Core Stack | React/Vite, FastAPI, tiny Hugging Face model, PostgreSQL, Docker Compose |
| Scope      | Minimal live demo focused on workflow, not feature depth                 |

#### 1. Product Overview

Ticket Analyzer is a small full-stack application that accepts a support ticket, analyzes its sentiment using a tiny Hugging Face model, stores the ticket in PostgreSQL, and shows the ticket history in a simple frontend.

The purpose is to demonstrate the complete engineering path: PRD, frontend, backend, AI model integration, database persistence, Docker images, DockerHub, GitHub, and deployment on a Poridhi Lab VM.

#### 2. Demo Objective

- Turn a PRD into a working full-stack application using Puku Editor.
- Use Puku CLI for local testing, Docker debugging, and deployment commands.
- Containerize the frontend and backend separately.
- Push source code to GitHub and images to DockerHub.
- Deploy the same application on a Poridhi Lab VM using Docker Compose.

#### 3. Minimal Scope

| Area     | Included                      | Not Included              |
|----------|-------------------------------|---------------------------|
| Frontend | One page form and ticket list | Authentication, dashboard |
| Backend  | Three API endpoints           | Complex service layers    |
| AI       | Tiny sentiment model          | Fine-tuning or LLM agents |

|            |                        |                         |
|------------|------------------------|-------------------------|
| Database   | PostgreSQL persistence | Migrations or analytics |
| Deployment | Docker Compose on VM   | Kubernetes or CI/CD     |

## 4. User Flow

1. User opens the Ticket Analyzer frontend.
2. User enters a ticket title, message, and optional category.
3. Frontend sends the ticket to the FastAPI backend.
4. Backend runs sentiment analysis using the tiny model.
5. Backend saves the result in PostgreSQL.
6. Frontend refreshes the ticket list.

## 5. Features

| Feature        | Requirement                                            |
|----------------|--------------------------------------------------------|
| Submit ticket  | User can submit title, message, and optional category. |
| Analyze ticket | Backend returns sentiment label and confidence score.  |
| Persist ticket | Every ticket is saved in PostgreSQL.                   |
| View tickets   | Frontend displays latest saved tickets.                |
| Health check   | Backend exposes GET /health.                           |

## 6. Architecture

```

React Frontend
  -> FastAPI Backend
      -> Tiny Hugging Face Sentiment Model
      -> PostgreSQL Database

```

- frontend: React production build served by Nginx on port 3000.
- backend: FastAPI served by Uvicorn on port 8000.
- db: PostgreSQL with a named Docker volume for persistence.

## 7. API Requirements

| Method | Endpoint | Purpose                              |
|--------|----------|--------------------------------------|
| GET    | /health  | Return backend status.               |
| POST   | /tickets | Create ticket and analyze sentiment. |

|     |          |                     |
|-----|----------|---------------------|
| GET | /tickets | List saved tickets. |
|-----|----------|---------------------|

### Example Request

POST /tickets

```
{
  "title": "Lab VM issue",
  "message": "My lab VM is not opening before the deadline.",
  "category": "lab"
}
```

### Example Response

```
{
  "id": 1,
  "title": "Lab VM issue",
  "sentiment": "NEGATIVE",
  "confidence": 0.999,
  "created_at": "2026-05-20T10:30:00"
}
```

## 8. Data Model

| Field      | Type      | Notes                  |
|------------|-----------|------------------------|
| id         | integer   | Primary key            |
| title      | string    | Ticket title           |
| message    | text      | Ticket body            |
| category   | string    | Optional category      |
| sentiment  | string    | Model output label     |
| confidence | float     | Model confidence score |
| created_at | timestamp | Created by backend     |

## 9. AI Model Requirement

Use a very small Hugging Face sentiment classification model for the live demo. Suggested model:

distilbert-base-uncased-finetuned-sst-2-english

- Download model weights during Docker build, not at runtime. The backend Dockerfile must include a step that runs `from_pretrained()` so the weights are baked into the image.
- Pin the HuggingFace cache directory with `HF_HOME` and set `TRANSFORMERS_OFFLINE=1` at runtime so the container fails loudly during dry-run if weights are not present, instead of silently downloading on stage.

- Load the model into memory once at backend startup (not lazily on the first request) so the first ticket submission during the demo is fast.
- Use the model only to demonstrate AI integration inside a real app.
- Pre-build and push the backend image (with weights baked in) to DockerHub before the session.

## 10. Repository Structure

```
ticket-analyzer/
├── PRD.md
├── README.md
├── docker-compose.yml
├── backend/
│   ├── Dockerfile
│   └── app/
├── frontend/
│   ├── Dockerfile
│   └── src/
```

## 11. Environment Variables

| Service  | Variable             | Example                                                                            |
|----------|----------------------|------------------------------------------------------------------------------------|
| backend  | DATABASE_URL         | postgresql://postgres:postgres@db:5432/ticket_db                                   |
| backend  | MODEL_NAME           | distilbert-base-uncased-finetuned-sst-2-english                                    |
| backend  | HF_HOME              | /opt/hf-cache                                                                      |
| backend  | TRANSFORMERS_OFFLINE | 1                                                                                  |
| frontend | VITE_API_BASE_URL    | /api (Nginx in the frontend container reverse-proxies /api to the backend service) |

## 12. Docker and Deployment Requirements

- Backend image: <dockerhub-username>/ticket-analyzer-backend:v1
- Frontend image: <dockerhub-username>/ticket-analyzer-frontend:v1
- Database image: official postgres image
- Local command: docker compose up --build
- Lab VM command: docker compose up -d using DockerHub images
- Backend Dockerfile installs torch from the CPU-only index (--index-url <https://download.pytorch.org/whl/cpu>) to keep the image around 600-800 MB instead of 2+ GB.
- Backend Dockerfile bakes the model weights into the image using from\_pretrained() during build, with HF\_HOME set so the runtime container finds the cache.



- Backend creates the tickets table on startup via SQLAlchemy `Base.metadata.create_all(engine)` so a fresh Postgres volume works without manual migrations.
- `docker-compose.yml` uses `depends_on` with a Postgres healthcheck so the backend waits for the database to be ready before starting (prevents connection-refused crashes on cold boot).
- Frontend Nginx reverse-proxies `/api` to the backend service on the internal Docker network. This removes the need for CORS configuration and lets the same image work on localhost and on the Poridhi Lab VM without rebuilding.
- If a direct cross-origin setup is used instead of the reverse proxy, the backend must enable FastAPI `CORSMiddleware` allowing the frontend origin.

### 13. Acceptance Criteria

- Frontend opens successfully.
- `GET /health` returns status ok.
- User can submit a ticket from the frontend.
- Backend analyzes ticket sentiment.
- Ticket is saved in PostgreSQL.
- Refreshing the page still shows saved tickets.
- Images are pushed to DockerHub and code is pushed to GitHub.
- Application runs successfully on the Poridhi Lab VM.
- Dry-run: backend container starts with no network access to `huggingface.co` (proves model weights are baked into the image, not downloaded at runtime).
- Dry-run: with a fresh Postgres volume, the tickets table is created automatically on first start and `POST /tickets` succeeds.
- Dry-run: frontend successfully submits a ticket from a browser that is not on the VM's localhost (validates the reverse-proxy / API base URL setup).
- Dry-run: sentiment output uses `POSITIVE/NEGATIVE` labels (not `LABEL_0/LABEL_1`), confirming the real `distilbert-sst-2` model is being used.